

REVERSE GUI WINDOWS

Từ một thao tác trên giao diện đến logic thật trong binary

EVENT	HANDLER	LOGIC	STATE
-------	---------	-------	-------

Playbook thực hành cho IDA và x64dbg: Win32, dialog, WTL/MFC, Qt, .NET, worker thread và game loop.

Mục lục

Cách dùng tài liệu này	4
Phạm vi	4
Ký hiệu	4
1. Mô hình đúng: chương trình GUI không chạy như một bài thuật toán	5
Luồng khởi động điển hình	5
Ba lớp cần tách	5
2. Phân loại binary trước khi đào code	6
Kiểm tra nhanh	6
3. Từ PE EntryPoint đến cửa sổ và dispatcher	7
Đường tắt 1: RegisterClassEx	7
Đường tắt 2: CreateWindowEx/DialogBoxParam	7
Nhận prototype WndProc trong decompiler	7
4. Message map: biến số hex thành sự kiện có nghĩa	8
WM_COMMAND	8
WM_NOTIFY	8
5. Resource là bản đồ nối giao diện với code	9
Quy trình	9
Nếu resource không còn	9
6. Workflow static trong IDA: tìm đủ, không đọc lan man	10
Vòng 1 - Inventory	10
Vòng 2 - Anchor và xref	10
Vòng 3 - Rename theo tầng bằng chứng	10
7. Workflow dynamic trong x64dbg: bắt đúng event	11
Công thức 8 bước	11
Breakpoint theo hướng từ kết quả đi ngược	11
8. Cách tách handler khỏi logic thật	12
Dấu hiệu của core logic	12
Data flow tối thiểu phải ghi	12
9. Worker thread, thread pool và custom message	13
Anchor tạo công việc	13
Case WTL trong tài liệu nguồn	13
10. WTL/MFC/Qt/.NET: khi không có switch(msg) đẹp	14
WTL/MFC	14
Qt	14
.NET	14
11. Game GUI: WndProc không phải main logic	15
Phân loại các hàm mỗi frame	15
Tìm win condition đúng cách	15

12. Case study A: lần nút Login tới ValidatePassword	16
Bước 1 - Lập bảng quan sát	16
Bước 2 - Từ resource/control ID	16
Bước 3 - Tách UI khỏi logic	16
Bước 4 - Chứng minh	16
Bước 5 - Theo thuật toán	16
13. Case study B: custom message và kết quả từ worker	17
Cách dựng lại	17
Điểm mấu chốt	17
14. Stripped binary: dựng semantics từ bằng chứng nhỏ	18
Bộ bằng chứng nên khai thác	18
Cách rename bền vững	18
Object reconstruction	18
15. Khi static và dynamic không khớp	19
Subclassing và thay WndProc	19
Callback resolve động	19
16. Playbook 60 phút cho một GUI binary mới	20
Mẫu evidence log	20
17. Checklist thực chiến và template ghi chú	21
Checklist trước khi kết luận	21
Template bản đồ hàm	21
Template câu hỏi cho phiên tiếp	21
18. Bảng tra nhanh	22
Registers x64 tại các điểm thường dùng	22
Anchor theo mục tiêu	22
19. Bài tập tự luyện	23
Bài 1 - Raw Win32	23
Bài 2 - Worker	23
Bài 3 - Framework	23
Bài 4 - Mini game loop	23
Tiêu chí tự chấm	23
20. Nguồn và phạm vi tham khảo	24
Nguồn chính	24
Lưu ý về số trang	24
Giới hạn	24

Cách dùng tài liệu này

Tài liệu này không dạy m đọc toàn bộ một executable. Nó dạy m chọn một hành động cụ thể trên GUI, bắt đúng đường code mà hành động đó kích hoạt, rồi dựng lại logic đủ để trả lời câu hỏi reverse. Cách tiếp cận này phù hợp nhất với binary hợp pháp, mẫu CTF, phần mềm do m sở hữu hoặc mẫu malware trong lab cô lập.

Mục tiêu cuối cùng

Sau mỗi phiên reverse, m phải viết được một chuỗi có bằng chứng: thao tác X -> message/callback Y -> handler Z -> hàm logic A -> biến trạng thái B -> hiệu ứng C.

Nếu mới bắt đầu, hãy đọc phần 1 đến 10 theo thứ tự. Khi đang xử lý một mẫu thật, dùng chương 16 như playbook 60 phút và chương 17 như checklist bàn giao.

Phạm vi

- Native Win32 x86/x64: window procedure, dialog procedure, message pump, control ID và resource.
- Framework native: WTL/ATL, MFC, Qt; cách nhận ra khi dispatcher bị framework che đi.
- .NET GUI: WinForms/WPF; khi nào nên rời IDA để dùng dnSpyEx hoặc ILSpy.
- Worker thread, thread pool, timer, callback và state machine.
- Game Windows: message pump chỉ xử lý cửa sổ; logic chính thường nằm ở main/update loop.

Ký hiệu

Ký hiệu	Ý nghĩa
Anchor	Một điểm bám có tên hoặc hành vi để nhận ra: API, string, resource ID, message constant.
Handler	Hàm nhận event/message và quyết định gọi logic nào.
Logic	Phần kiểm tra, giải mã, xử lý file/network, thuật toán, luật game hoặc thay đổi state.
State	Dữ liệu tồn tại qua nhiều event/frame: login state, current level, HP, mode, object field, global.
Evidence	Breakpoint hit, xref, call stack, giá trị thanh ghi/bộ nhớ hoặc thay đổi đầu ra xác nhận giả thuyết.

1. Mô hình đúng: chương trình GUI không chạy như một bài thuật toán

Trong console app, m thường lần từ `main()` xuống. GUI app lại chủ yếu phản ứng với message, callback và tác vụ bất đồng bộ. Vì vậy có thể không tồn tại một hàm duy nhất chứa toàn bộ logic.



Luồng khởi động điển hình

```

PE EntryPoint
-> CRT startup (WinMainCRTStartup / wWinMainCRTStartup)
-> WinMain / wWinMain
-> init config, COM, resources, objects
-> RegisterClassEx / CreateWindowEx hoặc DialogBoxParam
-> message pump
    GetMessage / PeekMessage
    TranslateMessage
    DispatchMessage
-> WndProc / DialogProc / framework dispatcher
  
```

Trong Windows 10 System Programming, Part 1, Pavel Yosifovich phân biệt entry của console và GUI: linker chọn `WinMainCRTStartup` hoặc `wWinMainCRTStartup` rồi runtime mới gọi `WinMain/wWinMain`. Cuốn sách cũng nhấn mạnh GUI thread phải thường xuyên kiểm tra message queue bằng `GetMessage` hoặc `PeekMessage`; nếu bỏ queue quá lâu, cửa sổ sẽ thành Not Responding [1, tr. 90-91, 105-109].

Câu hỏi dẫn đường

Đừng hỏi: program chạy tiếp đi đâu? Hãy hỏi: khi tao bấm nút X, code nào chạy; code đó đọc/ghi state nào; hiệu ứng nào chứng minh điều đó?

Ba lớp cần tách

Lớp	Ví dụ	Giá trị khi reverse
Plumbing GUI	<code>GetMessage</code> , <code>DispatchMessage</code> , <code>DefWindowProc</code> , vẽ control	Cho biết đường phân phối, thường không phải logic đích.
Handler	<code>OnLogin</code> , <code>OnOpen</code> , timer callback, slot Qt	Cửa ngõ từ event vào code của chương trình.
Logic/state	<code>ValidateKey</code> , <code>ParseFile</code> , <code>BossStateMachine</code> , session field	Nơi trả lời câu hỏi reverse.

2. Phân loại binary trước khi đào code

Quyết định công cụ và anchor phụ thuộc framework. Năm phút phân loại đầu tiên có thể tiết kiệm hàng giờ follow sai dispatcher.

Kiểm tra nhanh

- 1 Mở PE-bear hoặc CFF Explorer: xem Machine, Subsystem, Imports, Sections, .NET/CLR directory, TLS và resource.
- 2 Mở Detect It Easy: xem compiler, packer/protector, signature và framework gợi ý.
- 3 Trong IDA: xem Imports, Strings, Functions; ghi lại DLL và API đặc trưng.
- 4 Chạy mẫu trong VM: ghi lại tất cả cửa sổ, nút, menu, dialog, lỗi và hành động tạo thay đổi quan sát được.

Dấu hiệu	Khả năng	Công cụ/anchor ưu tiên
user32 + RegisterClassEx + CreateWindowEx + DefWindowProc	Raw Win32 hoặc wrapper mỏng	IDA/x64dbg; lần lpfnWndProc.
DialogBoxParam/CreateDialogParam + RT_DIALOG	Dialog-based Win32/WTL/MFC	Resource ID, DialogProc, WM_COMMAND.
mfc140*.dll, AfxWinMain, AfxWndProc	MFC dynamic link	Message map, command routing, strings/resource.
Qt5Widgets/Qt6Widgets, QObject, meta-object strings	Qt	Signals/slots, connect, eventFilter, vtable.
CLR header, mscoree, _CorExeMain	.NET WinForms/WPF	dnSpyEx/ILSpy; event subscription và XAML.
DirectX/OpenGL/SDL + PeekMessage + loop liên tục	Game/engine	Main/update loop, input, state machine, render boundary.
Ít imports, section lạ, entropy cao, entry stub	Packed/protected	Xác nhận unpacking trước khi phân tích logic.

Sai lầm phổ biến

Thấy user32.dll không có nghĩa chắc chắn là raw Win32. MFC, WTL, Qt và cả loader/plugin đều có thể gọi User32. Hãy kết hợp imports, resource, strings, RTTI và hành vi runtime.

3. Từ PE EntryPoint đến cửa sổ và dispatcher

EntryPoint là chỗ loader chuyển quyền điều khiển, thường là CRT startup chứ không phải hàm do lập trình viên viết. Dùng nó để hiểu startup, không mặc định coi đó là logic chính.

Đường tắt 1: RegisterClassEx

RegisterClassExW nhận con trỏ tới WNDCLASSEXW. Trường lpfnWndProc chứa địa chỉ window procedure. Ở x64, tham số đầu tiên nằm trong RCX; khi breakpoint hit, dump cấu trúc và lấy qword tại offset +8.

WNDCLASSEXW x64	Offset	Ý nghĩa reverse
cbSize / style	+0 / +4	Kiểm tra có đúng cấu trúc không.
lpfnWndProc	+8	Địa chỉ dispatcher cần đặt breakpoint/rename.
hInstance	+24	Module sở hữu class/resource.
lpszMenuName	+56	Tên hoặc ID menu.
lpszClassName	+64	Tên class cửa sổ, có thể tìm bằng string/xref.

```
bp user32!RegisterClassExW
run
; x64: RCX = WNDCLASSEXW*
; [RCX+08] = lpfnWndProc
; [RCX+40] = lpszClassName (0x40 = 64 decimal)
```

Đường tắt 2: CreateWindowEx/DialogBoxParam

- xref CreateWindowExW: tìm class name, title, parent, control ID và object pointer được lưu sau khi tạo.
- xref DialogBoxParamW/CreateDialogParamW: tham số callback là DialogProc; resource template cho biết control IDs.
- xref DefWindowProcW: caller thường là WndProc hoặc wrapper rất gần dispatcher.
- xref DispatchMessageW: cho biết message pump; đừng dừng ở đó nếu mục tiêu là logic nghiệp vụ.

Nhận prototype WndProc trong decompiler

```
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam)

// Windows x64 calling convention khi vào trực tiếp WndProc:
RCX = hwnd
RDX = msg
R8  = wParam
R9  = lParam
```

Dấu vân tay mạnh

Một hàm có 4 tham số, so sánh tham số thứ hai với 0x111/0x10/0x0F và cuối cùng gọi DefWindowProcW rất có khả năng là WndProc.

4. Message map: biến số hex thành sự kiện có nghĩa

Message không tự nói handler nào là quan trọng. Giá trị của nó chỉ cho m biết loại event; m vẫn phải giải mã wParam/lParam và kiểm tra nhánh được thực thi.

Message	Hex	Khi nào đáng theo
WM_CREATE	0x0001	Khởi tạo sau khi tạo window; lParam -> CREATESTRUCT.
WM_DESTROY	0x0002	Dọn dẹp, PostQuitMessage.
WM_PAINT	0x000F	Render UI; đôi khi lộ state nhưng thường không phải logic.
WM_CLOSE	0x0010	Đóng/cancel, save config, anti-exit.
WM_COMMAND	0x0111	Button, menu, accelerator; anchor cực quan trọng.
WM_TIMER	0x0113	Polling, periodic update, game/tool timer.
WM_NOTIFY	0x004E	Notification phức tạp từ common controls.
WM_INITDIALOG	0x0110	Khởi tạo dialog và object/control.
WM_KEYDOWN / WM_KEYUP	0x0100 / 0x0101	Hotkey, input state.
WM_LBUTTONDOWN / UP	0x0201 / 0x0202	Custom-drawn UI/game input.
WM_USER	0x0400	Mốc bắt đầu custom message nội bộ.
WM_APP	0x8000	Custom message theo app, thường dùng worker -> GUI.

WM_COMMAND

```
control_or_menu_id = LOWORD(wParam)
notification_code = HIWORD(wParam)
control_hwnd      = (HWND)lParam

if (msg == WM_COMMAND && LOWORD(wParam) == 1001)
    OnLoginClicked();
```

Trong x64dbg, nếu breakpoint đặt ở đầu WndProc: message ở RDX, wParam ở R8. Lấy ID bằng R8 & 0xFFFF; lấy notification bằng (R8 >> 16) & 0xFFFF. Với menu, lParam thường bằng 0.

WM_NOTIFY

```
typedef struct tagNMHDR {
    HWND    hwndFrom;
    UINT_PTR idFrom;
    UINT     code;
} NMHDR;

// x64 khi vào WndProc: R9 = lParam = NMHDR*
// [R9+00] hwndFrom, [R9+08] idFrom, [R9+10] code
```

Ghi chú

Cùng một control ID có thể tạo nhiều notification code. Đừng chỉ thấy ID rồi kết luận handler; kiểm tra cả HIWORD(wParam) hoặc NMHDR.code.

5. Resource là bản đồ nối giao diện với code

Với dialog/menu truyền thống, resource thường cho m tên caption, layout và số ID. Sau đó search số ID trong disassembly/decompiler để tìm nhánh handler.

Quy trình

- 1 Mở Resource Hacker hoặc PE-bear; liệt kê RT_DIALOG, RT_MENU, STRINGTABLE, VERSION và accelerator.
- 2 Chụp lại cặp caption - numeric ID: Login = 1001, Cancel = 1002, Password edit = 1004...
- 3 Trong IDA search immediate value của ID; xem xref và điều kiện xung quanh, đặc biệt dưới WM_COMMAND/WM_NOTIFY.
- 4 Đặt breakpoint tại nhánh hoặc callee; thao tác đúng control để xác nhận.
- 5 Rename handler theo bằng chứng: OnLoginClicked, không dùng tên quá tự tin như DecryptMasterPassword khi chưa chứng minh.

```
Resource: PUSHBUTTON "Login", 1001, ...

Decompiler:
if ((unsigned short)wParam == 1001) {
    sub_140014A20(ctx);
}

Evidence: click Login -> breakpoint sub_140014A20 hit
Rename: sub_140014A20 -> OnLoginClicked
```

Nếu resource không còn

- Control có thể được tạo động bằng CreateWindowEx; ID nằm ở tham số hMenu đối với child window.
- UI có thể custom-draw; khi đó lần input message, hit-test, coordinate và object/vtable.
- Qt/WPF có thể dùng dữ liệu UI riêng; tìm objectName, XAML/BAML, meta-object string hoặc constructor.
- Protector có thể mã hóa string/resource; dùng runtime breakpoint và memory map sau khi giải mã.

6. Workflow static trong IDA: tìm đủ, không đọc lan man

Static analysis dùng để xây bản đồ và đặt giả thuyết. Mỗi rename nên có lý do; mỗi nhánh quan trọng nên có kế hoạch xác nhận động.

Vòng 1 - Inventory

- Imports: User32, Comctl32, Shell32, Ole32/COM, networking, crypto, file, registry, thread APIs.
- Strings: title cửa sổ, caption nút, thông báo lỗi/thành công, path, URL, format string, class name.
- Resources: dialog/menu/string/icon/version; ghi numeric ID.
- Functions: thunk/import wrapper, CRT noise, constructors, callback-like prototypes, giant switch/state machine.

Vòng 2 - Anchor và xref

Mục tiêu	Anchor bắt đầu	Đi ngược/đi xuôi
Nút Login/Check	Caption, control ID, GetDlgItemText, MessageBox	Từ input/output vào caller rồi tới compare/validator.
Open/Save file	GetOpenFileName, CreateFile, ReadFile, WriteFile	Tìm handler mở dialog, parser và state nhận dữ liệu.
Network action	WinHTTP/WinINet/WSA/cURL imports, URL string	Caller -> request builder -> parser -> UI update.
License/flag	success/failure strings, memcmp, hash/crypto APIs	Lần ngược tới input và điều kiện nhánh.
Boss/win condition	text win/lose, score/HP write, state switch	Theo biến state và writer, không chỉ reader/render.

Vòng 3 - Rename theo tầng bằng chứng

Mức	Ví dụ tên	Khi dùng
Chắc chắn	MainWndProc, OnLoginClicked	Breakpoint + message/control ID đã xác nhận.
Khả năng	MaybeValidateKey, CandidateWorker	Call shape/hành vi gợi ý nhưng thiếu kiểm chứng.
Trung tính	Handler_ID1001, Writer_PlayerState_30	Biết vai trò cấu trúc nhưng chưa biết semantics.

Quy tắc chống tự lừa

Tên hàm là một giả thuyết. Nếu bằng chứng mới mâu thuẫn, sửa tên ngay. Một tên sai nhưng đẹp có thể kéo cả buổi phân tích đi lệch.

7. Workflow dynamic trong x64dbg: bắt đúng event

Dynamic analysis trả lời câu hỏi quan trọng nhất: hành động nào thật sự khiến nhánh nào chạy. Làm trong VM snapshot, tắt shared folder/clipboard khi xử lý mẫu không tin cậy và không dùng dữ liệu thật.

Công thức 8 bước

- 1 Chọn đúng một hành động có đầu ra quan sát được, ví dụ bấm Login với password sai.
- 2 Đặt breakpoint ở anchor gần event: WndProc, DialogProc, GetDlgItemTextW, MessageBoxW hoặc handler nghi ngờ.
- 3 Run đến trạng thái ngay trước thao tác; xóa/disable breakpoint gây nhiễu.
- 4 Thực hiện đúng một thao tác; ghi thread ID và call stack khi dừng.
- 5 Giải mã message/control ID hoặc tham số API; xác nhận breakpoint liên quan chứ không phải background noise.
- 6 Step out/return về code module của chương trình nếu đang ở User32/ntdll.
- 7 Theo các call không phải UI; đánh dấu input buffer, return value và branch quyết định success/failure.
- 8 Lặp lại với input đối chứng; so call trace, branch và state để loại giả thuyết sai.

Breakpoint theo hướng từ kết quả đi ngược

Dấu hiệu GUI	Breakpoint gợi ý	Thông tin lấy
Message box success/fail	user32!MessageBoxW	RCX hwnd, RDX text, R8 caption; call stack về caller.
Đọc edit control	user32!GetDlgItemTextW / GetWindowTextW	ID control, buffer đích, caller.
Ghi text lên UI	SetDlgItemTextW / SetWindowTextW	String kết quả và caller.
Tạo dialog/window	DialogBoxParamW / CreateWindowExW	Callback, template/class/title.
Gửi custom message	PostMessageW / SendMessageW	hwnd, message, wParam, lParam; worker -> GUI.

Ví dụ: break user32!MessageBoxW (x64)

```
RCX = hwnd
RDX = lpText    -> follow in dump, xem Unicode
R8  = lpCaption -> follow in dump
R9  = uType
```

Sau khi xác nhận đúng message box:

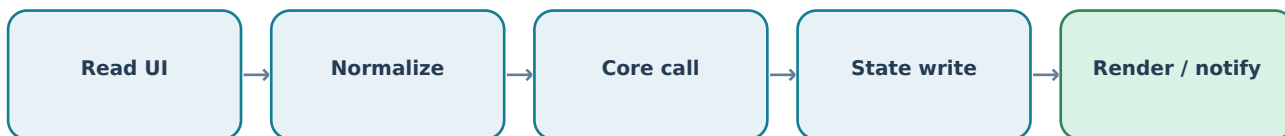
```
xem Call Stack -> chọn frame đầu tiên thuộc module target
đặt breakpoint tại call site -> chạy lại -> lộn ngược đi đầu kiện
```

Nếu breakpoint hit quá nhiều

Dùng conditional breakpoint theo thread ID, message value, module return address hoặc control ID. Hoặc đặt breakpoint tại call site trong module thay vì API toàn hệ thống.

8. Cách tách handler khỏi logic thật

Một handler thường gồm bốn pha: lấy input từ UI, chuẩn hóa, gọi logic, rồi cập nhật UI. Mục tiêu là xác định ranh giới giữa plumbing và logic.



```

OnLoginClicked(ctx)
  GetDlgItemTextW(...) // plumbing: lấy input
  TrimAndNormalize(buf) // preparation
  ok = sub_140018450(ctx, buf) // candidate logic
  ctx->authenticated = ok // state
  SetDlgItemTextW(...) // plumbing: hiển thị kết quả
  
```

Dấu hiệu của core logic

- Không phụ thuộc trực tiếp HWND/control hoặc chỉ nhận buffer/object đã chuẩn hóa.
- Có loop, compare, parser, crypto/hash, state transition hoặc tạo side effect file/network.
- Return value điều khiển nhánh success/failure của handler.
- Được gọi từ nhiều handler/callback khác nhau.
- Ghi field/global được đọc ở render, timer hoặc event sau.

Data flow tối thiểu phải ghi

Câu hỏi	Ví dụ câu trả lời tốt
Input đến từ đâu?	Edit control ID 1004 -> buffer UTF-16 stack tại rbp-220h.
Biến đổi ở đâu?	NormalizeEmail lowercase, sau đó SHA-256.
Quyết định ở đâu?	cmp eax, 0 tại 1400189F2; JZ đi nhánh fail.
State nào đổi?	ctx+0x58 đổi 0 -> 1; render đọc lại field này.
Hiệu ứng nào xác nhận?	Caption chuyển Authorized; nút Start enabled.

9. Worker thread, thread pool và custom message

Handler có thể chỉ lên lịch công việc rồi return. Khi đó logic thật chạy ở thread khác; kết quả quay lại GUI bằng PostMessage, SendMessage, callback, future hoặc shared state.

```
OnScanClicked
-> allocate WorkContext
-> CreateThread / _beginthreadex / TrySubmitThreadpoolCallback
-> return

WorkerProc(ctx)
-> scan / parse / network / decrypt
-> ctx->result = ...
-> PostMessage(hwnd, WM_APP + 7, status, ctx)

WndProc: case WM_APP + 7
-> update progress/result controls
```

Anchor tạo công việc

Cơ chế	Anchor	Callback nằm ở đâu
Thread trực tiếp	CreateThread, _beginthreadex	Tham số start routine.
Thread pool work	CreateThreadpoolWork, TrySubmitThreadpoolCallback	Callback argument.
Queue work	QueueUserWorkItem	Function pointer argument.
Timer	SetTimer, CreateThreadpoolTimer	WndProc WM_TIMER hoặc timer callback.
Async I/O	ReadFileEx, completion port, threadpool I/O	Completion routine/worker.

Tài liệu nguồn cảnh báo GUI thread không nên chờ vô hạn bằng WaitForSingleObject/WaitForMultipleObjects vì sẽ làm UI treo; nếu cần vừa chờ vừa xử lý message, Windows có MsgWaitForMultipleObjects và message pump xen kẽ [1, tr. 395-398]. Đây là dấu hiệu quan trọng khi reverse một UI treo hoặc handler chuyển việc sang worker.

Case WTL trong tài liệu nguồn

Ví dụ Single Instance tạo mutex trong WinMain. Instance thứ hai tìm cửa sổ cũ rồi gửi custom message WM_USER + 100 bằng PostMessage. WTL map message đó tới OnNotifyInstance; handler lấy PID từ wParam và cập nhật list box [1, tr. 45-48]. Đây là mẫu hoàn chỉnh của chuỗi event -> custom message -> message map -> handler -> UI effect.

Thực chiến

Nếu click button không dẫn tới logic, đặt breakpoint ở API tạo thread/submit work, lấy callback address, rồi đặt breakpoint ở chính callback. Ghi thread ID để tránh lẫn GUI thread với worker.

10. WTL/MFC/Qt/.NET: khi không có switch(msg) đẹp

Framework thường gom message/callback qua dispatcher chung, bảng ánh xạ, vtable hoặc metadata. Mục tiêu vẫn giống nhau: lấy một event cụ thể làm đầu mối và tìm callback cuối cùng thuộc code ứng dụng.

Framework	Cơ chế	Chiến thuật
WTL/ATL	Macro message map; thường static link	Resource/control ID, custom message, callback signature, compare constants.
MFC	Message map + command routing	AfxWndProc/CCmdTarget, resource IDs, handler prototype, RTTI nếu còn.
Qt	signals/slots, meta-object, event(), eventFilter()	Qt DLL/version, objectName/string, connect, qt_metacall, vtable.
WinForms	Delegate/event subscription	dnSpyEx/ILSpy; tìm += Click, InitializeComponent, event handler.
WPF	XAML/BAML, routed events, binding/command	Decompile managed code; command Execute, code-behind, ViewModel.

WTL/MFC

- Search numeric control ID trước; macro đã biến mất nhưng constant và handler call vẫn thường còn.
- Đặt breakpoint trên handler nghi ngờ rồi click control; nếu hit, rename theo event trước khi hiểu sâu logic.
- Dispatcher chung có thể hit liên tục; dùng call stack, this pointer và HWND để lọc instance đúng.

Qt

- Tìm string của objectName, menu action, button text; xref về constructor/Ui setup.
- Theo chỗ kết nối signal-slot hoặc metadata qt_metacall; slot cuối cùng mới là handler ứng dụng.
- Nếu binary stripped/static Qt, dynamic breakpoint ở side-effect API thường nhanh hơn duyệt toàn bộ meta-object.

.NET

- Nếu có CLR header, ưu tiên dnSpyEx/ILSpy vì decompiler managed giữ class, method, metadata và event subscription tốt hơn IDA.
- WinForms: vào InitializeComponent, tìm button.Click += handler.
- WPF: kiểm tra XAML/BAML, RoutedEvent, ICommand.Execute và binding tới ViewModel.

11. Game GUI: WndProc không phải main logic

Game có cửa sổ Windows nhưng thường chạy loop liên tục. WndProc chủ yếu xử lý close, focus, resize và một phần input; HP, enemy, boss, collision và win condition nằm trong update/state machine.

```
WinMain
-> CreateWindow
-> InitGraphics / InitAudio / LoadAssets
-> while (!ShouldQuit()) {
    PumpWindowMessages();
    ReadInput();
    UpdateGame();          // physics, AI, state transitions
    CheckWinLose();
    Render();
}
```

Phân loại các hàm mỗi frame

Dấu hiệu	Vai trò có thể	Cách xác nhận
Gọi PeekMessage/DispatchMessage	Pump OS message	Breakpoint hit nhiều lần; không đổi game state chính.
Đọc GetKeyState/DirectInput/XInput	Input	Nhấn/thả phím và xem buffer/state.
Loop object, delta time, nhiều float	Update/physics/AI	Freeze/step một frame; xem position/HP writer.
DirectX Present/SwapBuffers	Cuối render frame	Dùng làm ranh giới frame và backtrace caller.
Switch enum/state + spawn/despawn	State machine/level/boss	Track field state qua transition.

Tìm win condition đúng cách

- 1 Xác định state nhìn thấy: HP bar, phase, current level, boss alive, score, dialog win/lose.
- 2 Tìm writer của state bằng hardware breakpoint/watchpoint; reader trong render thường chỉ hiển thị, không quyết định.
- 3 Gây transition hợp lệ trong game; ghi call stack khi field đổi.
- 4 Tìm state machine quyết định transition; phân biệt health == 0 với điều kiện scripted/phase/quest.
- 5 Chỉ sau khi hiểu invariant mới thử patch trong mẫu CTF/lab; patch mỗi HP có thể vi phạm script và tạo lỗi.

Liên hệ mẫu space shooter

Loop kiểu while (!sub_should_quit()) { sub_A(); sub_B(ctx); sub_C(ctx); } là tín hiệu mạnh của main loop. Hãy đo từng call bằng breakpoint theo frame và side effect, rồi rename ProcessMessages / Update / Render dưới dạng giả thuyết.

12. Case study A: lần nút Login tới ValidatePassword

Giả sử mẫu native dialog có Username, Password, Login và hai message box. Mục tiêu: tìm thuật toán kiểm tra password, không cần hiểu toàn bộ chương trình.

Bước 1 - Lập bảng quan sát

Thử nghiệm	Input	Output	Ý nghĩa
A	user=a, pass=a	Invalid password	Baseline fail.
B	user rỗng	Enter username	Có validation trước password.
C	pass dài 40	Invalid password	Không crash; có giới hạn/copy an toàn.

Bước 2 - Từ resource/control ID

```
IDC_LOGIN = 1001
IDC_USER  = 1003
IDC_PASS  = 1004

Search 1001 -> nhánh WM_COMMAND -> call sub_140013420
Dynamic click Login -> sub_140013420 hit
Rename -> OnLoginClicked
```

Bước 3 - Tách UI khỏi logic

```
OnLoginClicked(ctx):
    GetDlgItemTextW(hwnd, 1003, user, 128)
    GetDlgItemTextW(hwnd, 1004, pass, 128)
    if (!sub_140018450(ctx, user, pass))
        MessageBoxW(..., L"Invalid password", ...)
    else
        EnableWindow(startButton, TRUE)
```

Bước 4 - Chứng minh

- Breakpoint tại sub_140018450: chỉ hit khi username không rỗng và bấm Login.
- Xem tham số: RCX=ctx, RDX=user buffer, R8=pass buffer.
- Step over return; EAX=0 dẫn tới message fail, EAX=1 dẫn tới EnableWindow.
- Đặt tên ValidateCredentials; chưa gọi là DecryptPassword nếu chưa thấy decrypt.

Bước 5 - Theo thuật toán

Bây giờ mới đi sâu vào hàm validator: theo data flow của pass, loop/transform, constant, hash/compare, dữ liệu expected và nhánh trả về. Dùng hai input khác nhau để kiểm tra byte nào ảnh hưởng output. Kết luận chỉ dựa trên branch/return và buffer quan sát được.

13. Case study B: custom message và kết quả từ worker

Giả sử bấm Scan nhưng handler return gần như ngay lập tức. Sau vài giây list box hiện kết quả. Đây là pattern bất đồng bộ.

```
WM_COMMAND / ID_SCAN
-> OnScanClicked
    -> alloc ScanContext
    -> CreateThread(WorkerProc, ctx)
    -> return

WorkerProc
-> EnumerateFiles
-> AnalyzeItem
-> PostMessageW(hwnd, 0x8007, count, resultPtr)

WndProc / msg 0x8007
-> InsertResultRows
-> SetWindowTextW(status)
```

Cách dựng lại

- 1 Breakpoint CreateThread chỉ trong lần click Scan; lấy start routine từ tham số thứ ba (R8 trên x64).
- 2 Đặt breakpoint tại WorkerProc; ghi thread ID khác GUI thread.
- 3 Tìm PostMessageW trong worker; ghi message 0x8007 và ý nghĩa wParam/lParam bằng quan sát.
- 4 Ở WndProc, bắt msg == 0x8007; xác nhận nhánh cập nhật list box.
- 5 Dựng object ScanContext: field nào là hwnd, count, result pointer, cancel flag, progress.

Điểm mấu chốt

Không được kết luận handler Scan là logic chính chỉ vì đó là điểm bắt đầu. Handler là producer của công việc; WorkerProc mới chứa core logic, còn custom-message handler là consumer của kết quả.

Pattern đối xứng

Tìm cả hai chiều: submit work -> callback và PostMessage -> message handler. Khi nối được hai nửa, m có bản đồ hoàn chỉnh qua ranh giới thread.

14. Stripped binary: dựng semantics từ bằng chứng nhỏ

Strip xóa tên symbol, không xóa cấu trúc chương trình. Call graph, API, constant, resource, string, calling convention, object layout và hành vi runtime vẫn còn.

Bộ bằng chứng nên khai thác

Bằng chứng	Nó cho biết gì	Độ tin cậy
Breakpoint theo event	Hàm thực sự chạy khi thao tác	Cao nếu thí nghiệm cô lập.
API/xref	Loại side effect hoặc plumbing	Trung bình; API có thể bọc nhiều tầng.
String/resource ID	Ngữ cảnh UI và nhánh	Cao khi liên kết với runtime hit.
Data writer/readers	State và lifetime	Cao; cần phân biệt pointer alias.
Prototype/calling convention	Vai trò callback và tham số	Trung bình đến cao.
Pattern compiler	switch, vtable, RTTI, exception	Trung bình; dễ nhầm nếu tối ưu mạnh.

Cách rename bền vững

```

sub_140020110 -> MainWndProc           [confirmed]
sub_140031840 -> Handler_Command_1001 [confirmed event]
sub_140043A20 -> MaybeValidateLicense [hypothesis]
qword_1400A3010 -> g_main_context     [likely]
field ctx+0x58 -> authenticated_flag  [confirmed writer+reader]
```

Object reconstruction

- Theo cùng một base pointer (thường RCX=this) và ghi các offset được đọc/ghi.
- Gom field theo type sử dụng: HWND/pointer/qword, flag byte, int counter, float, string object.
- Dùng constructor/destructor để xác định lifetime và vtable.
- Đổi một field trong debugger chỉ sau khi biết ai viết, ai đọc và invariant nào liên quan.

Không đọc từ đầu đến cuối

Một binary 30.000 hàm không yêu cầu hiểu 30.000 hàm. Một câu hỏi reverse tốt thường chỉ cần một lát cắt: một event, một đường call, vài field state và bằng chứng đầu ra.

15. Khi static và dynamic không khớp

Sai khác thường đến từ framework dispatch, thunk, optimization, callback đăng ký động, worker thread, code unpack lúc runtime hoặc m đang bắt nhầm instance/thread.

Triệu chứng	Nguyên nhân hay gặp	Bước tiếp theo
Click nhưng WndProc không hit	Sai window/child proc, subclassing, framework, hook	Dùng Spy++/WinSpy lấy HWND/TID; bắt DispatchMessage/call stack; kiểm tra SetWindowLongPtr.
WndProc hit liên tục	Nhiều paint/mouse/timer	Điều kiện RDX==message; lọc HWND/thread; break nhánh control ID.
Handler hit nhưng không có logic	Submit worker/callback	Break CreateThread/threadpool/PostMessage; theo context pointer.
String có nhưng xref rỗng	Runtime decode, indirect reference, resource	Memory search khi string hiển thị; hardware BP on access; inspect resource loader.
IDA decompile kỳ lạ	Sai function boundary/prototype, obfuscation	Sửa prototype/calling convention; xem assembly; xác định jump table.
Patch field gây crash/script error	Vi phạm invariant/state machine	Tìm writer hợp lệ và transition; patch condition/state cùng ngữ cảnh.
Breakpoint API không hit	API khác, syscall/wrapper, cached result	Bắt higher-level side effect; xem imports/dynamic GetProcAddress; thử call site.

Subclassing và thay WndProc

SetWindowLongPtrW(hwnd, GWLP_WNDPROC, newProc) có thể thay procedure sau khi tạo cửa sổ. Nếu lpfnWndProc ban đầu không nhận event mong đợi, tìm xref API này và chain CallWindowProc. Common controls/framework dùng pattern này khá nhiều.

Callback resolve động

Nếu imports thừa, tìm LoadLibrary/GetProcAddress và string API. Break GetProcAddress để ghi tên hàm và địa chỉ trả về, sau đó xref runtime hoặc đặt breakpoint tại địa chỉ.

16. Playbook 60 phút cho một GUI binary mới

Mục tiêu của phiên đầu không phải hiểu toàn bộ mẫu. Mục tiêu là chứng minh ít nhất một đường event -> logic -> state/effect và tạo bản đồ đủ sạch cho phiên sau.

Thời gian	Việc làm	Deliverable
0-5 phút	Hash/copy mẫu, snapshot VM, chạy quan sát, chọn một action	Một câu hỏi cụ thể và baseline behavior.
5-15 phút	PE/framework/resource/import/string inventory	Framework guess + 5-10 anchor.
15-25 phút	Tìm WndProc/DialogProc/message map hoặc managed handler	Một dispatcher/handler candidate.
25-40 phút	Dynamic breakpoint; click đúng action; lọc thread/message/ID	Breakpoint hit có call stack và args.
40-50 phút	Theo callee tới core call; ghi input/return/state	Một data-flow ngắn có địa chỉ.
50-60 phút	Lập với input đối chứng; rename/comment; viết kết luận	Evidence log + câu hỏi kế tiếp.

Mẫu evidence log

```
[Action] Click Login, user=a, pass=a
[Thread] GUI TID 0x1234
[Event] WndProc 140020110, msg=0x111, LOWORD(wParam)=1001
[Handler] 140031840 (OnLoginClicked)
[Input] GetDlgItemTextW IDs 1003/1004
[Core] 140043A20(ctx, user, pass), returns EAX=0
[Branch] 1400319F2 JZ -> fail
[State] ctx+0x58 unchanged (0)
[Effect] MessageBoxW "Invalid password"
[Next] Trace pass buffer inside 140043A20 with two controlled inputs
```

Stop condition tốt

Dừng một phiên khi m đã có bằng chứng lặp lại được và một câu hỏi tiếp theo rõ ràng. Đừng tiếp tục step vô định chỉ vì debugger vẫn đang mở.

17. Checklist thực chiến và template ghi chú

Checklist trước khi kết luận

- [] Đã xác định x86/x64, native/.NET, framework và tình trạng packed chưa?
- [] Đã chọn một action cụ thể thay vì mục tiêu mơ hồ 'tìm main logic' chưa?
- [] Đã xác nhận handler bằng breakpoint khi thực hiện đúng action chưa?
- [] Đã giải mã message + control ID/notification code chưa?
- [] Đã phân biệt GUI thread với worker thread chưa?
- [] Đã xác định input, core call, return/branch, state write và visible effect chưa?
- [] Đã chạy input đối chứng hoặc thao tác đối chứng chưa?
- [] Tên hàm/field có phản ánh đúng mức bằng chứng chưa?
- [] Nếu là game, đã tìm writer/state transition thay vì chỉ render reader chưa?
- [] Đã lưu address theo module offset/RVA để chịu được ASLR chưa?

Template bản đồ hàm

```

MODULE: sample.exe | arch: x64 | image base runtime: _____
FRAMEWORK: _____ | evidence: _____

RVA      NAME      CONFIDENCE  EVIDENCE
0x20110   MainWndProc   confirmed   DefWindowProc + msg BP
0x31840   Handler_Command_1001 confirmed   click Login hits
0x43A20   MaybeValidateLicense medium       return drives fail/success

STATE
base      offset  type      proposed name  writers / readers
ctx       +0x58   byte      authenticated  43A20 / render+Start

EVENT PATH
Action -> msg/callback -> handler -> core -> state -> effect

```

Template câu hỏi cho phiên tiếp

- Field ctx+0x58 được set ở những writer nào ngoài login?
- Custom message 0x8007 mang con trỏ hay integer ở lParam?
- Boss chuyển phase bởi HP, timer hay objective flag?
- sub_X là validator hay chỉ wrapper gọi service/crypto API?

18. Bảng tra nhanh

Registers x64 tại các điểm thường dùng

Điểm dừng	RCX	RDX	R8	R9
WndProc	HWND	UINT msg	WPARAM	LPARAM
MessageBoxW	HWND	text	caption	type
PostMessageW	HWND	msg	wParam	lParam
GetDlgItemTextW	HWND	control ID	buffer	max chars
RegisterClassExW	WNDCLASSEX*	-	-	-

Anchor theo mục tiêu

Muốn tìm	Bắt đầu từ
Handler nút/menu	WM_COMMAND 0x111, control/menu ID, caption resource.
Handler list/tree/tab	WM_NOTIFY 0x4E, NMHDR.idFrom/code.
Khởi tạo dialog	WM_INITDIALOG 0x110, DialogBoxParam callback.
Input text	GetDlgItemText/GetWindowText; buffer và call stack.
Output text	SetDlgItemText/SetWindowText/MessageBox.
Background work	CreateThread/_beginthreadex/threadpool/QueueUserWorkItem.
Worker trả kết quả	PostMessage/SendMessage/custom WM_APP/WM_USER.
Game loop	PeekMessage loop, DirectX Present, delta time, input/update calls.
State transition	Hardware breakpoint on write, writer call stack, switch/enum.

Một dòng cần nhớ

UI action -> message/callback -> handler -> core logic -> state -> effect. Nếu đang lạc, quay lại và hỏi mắt xích nào trong chuỗi này chưa có bằng chứng.

19. Bài tập tự luyện

Bài 1 - Raw Win32

Tự viết một app có hai button, một edit control và WndProc switch WM_COMMAND. Compile Release x64, strip symbol, rồi dùng chính workflow trong tài liệu để tìm lại handler. Deliverable: resource ID, WndProc RVA, handler RVA, ảnh breakpoint wParam.

Bài 2 - Worker

Sửa app để button tạo worker thread, sleep ngắn rồi PostMessage WM_APP+1 về GUI. Reverse lại mà không nhìn source. Deliverable: start routine, custom message, ý nghĩa wParam/lParam và context layout.

Bài 3 - Framework

Tạo WinForms hoặc Qt app có event Click. So sánh đường từ UI đến handler trong dnSpyEx/ILSpy hoặc IDA với raw Win32. Ghi rõ framework đã che dispatcher bằng cơ chế nào.

Bài 4 - Mini game loop

Tạo loop có ProcessMessages, Update và Render; thêm state win khi score đạt ngưỡng. Reverse binary Release và tìm writer của score/win state bằng hardware breakpoint.

Tiêu chí tự chấm

Mức	Kết quả
Chưa đạt	Chỉ tìm được WinMain/message pump nhưng chưa nối tới action cụ thể.
Đạt	Xác nhận handler và core call bằng breakpoint; giải thích input/output.
Tốt	Dựng state, writer/readers, worker boundary và kiểm chứng bằng thí nghiệm đối chứng.
Rất tốt	Ghi RVA, prototype, structure field, call graph tối giản và viết lại pseudocode sạch.

20. Nguồn và phạm vi tham khảo

Nguồn chính

[1] Pavel Yosifovich, Windows 10 System Programming, Part 1, bản ngày 2020-08-09. Các phần được dùng để dựng nền tảng: Running a Single Instance Process (tr. 45-48); trạng thái GUI process và message queue (tr. 90-91); CRT startup và WinMain/wWinMain (tr. 105-109); Waiting on GUI Threads (tr. 395-398); Thread Pools (từ tr. 401).

Lưu ý về số trang

Số trang trên là số trang in trong nội dung sách, có thể lệch với chỉ số trang của trình đọc PDF do phần bìa và mục lục.

Giới hạn

- Tên menu/phím tắt của IDA, x64dbg, PE-bear hoặc dnSpyEx có thể thay đổi theo phiên bản; nguyên tắc anchor/xref/call stack vẫn giữ nguyên.
- Calling convention/register trong bảng áp dụng cho Windows x64. Với x86, tham số thường nằm trên stack và callback có thể dùng stdcall/thiscall.
- Offset cấu trúc phụ thuộc kiến trúc và packing; luôn kiểm tra definition/sizeof thực tế.
- Tài liệu tập trung vào phân tích hợp pháp, CTF và lab. Không chạy mẫu không tin cậy trên máy thật hoặc môi trường có dữ liệu cá nhân.

Kết luận

Reverse GUI hiệu quả không phải là đọc hết binary. Nó là nghệ thuật chọn một event có thể lặp lại, bám vào callback/handler, theo data flow tới state và kiểm chứng bằng hiệu ứng quan sát được.